Research Computing Documentation

Slurm Tutorial 2: Scaling Up

Table of Contents

In Part 1 of this tutorial (slurm_quick_start_tutorial.html) you learned about the Slurm resource manager/job scheduler, how to tell Slurm what resources you need, and how to submit, monitor, and cancel your compute jobs. Now you are going to learn how to make the most of the cluster using parallel computation and GPUs.

0 - Prerequisites

This part of the tutorial covers some more advanced Slurm usage. If you haven't gone through Part 1 of this tutorial (slurm_quick_start_tutorial.html), we **strongly** recommend that you do so before starting Part 2.

1 - Review

1.1 - Slurm

Slurm is a resource manager and job scheduler. When you submit your job, Slurm finds a spot in the queue for you based on the resources you requested. The resources that you request are allocated to you and only you; no one else can use them while your job is running, so it's important to request only the resources that you **need** and that your code can **use**.

1.2 - Slurm Commands

Command	Example	Description
`my-accounts`	`$ my-accounts`	Show your Slurm accounts.
`sacct`	`$ sacct`	Show details about your recent jobs.
`sbatch`	`$ sbatch slurm_script.sh`	Submit a batch job.
`scancel`	`$ scancel --me` (all jobs) `$ scancel <jobID>` (single job)	Cancel a job.

Command	Example	Description
`scontrol`	`$ scontrol --help` (all options) `$ scontrol show job <jobID>` (job details)	Show details of a specific job.
`sinfo`	`$ sinfo` (general) `$ sinfo -o '%11P %5D %22N %4c %21G %7m %11l'` (detailed)	Show details about the cluster.
`sinteractive`	`$ sinteractive` (CPU only) `$ sinteractive --gres=gpu:<numGPUs>` (with GPUs)	Submit an interactive job.
`squeue`	`$ squeue` (all jobs) `$ squeue --me` (your jobs)	Monitor jobs.
`time-until-maintenance`	`$ time-until-maintenance`	Show upcoming maintenance windows (maintenance.html).

1.3 - Slurm Configurations

This section provides a very brief summary of configuration options for Slurm. Please review the full list in Part 1 of this tutorial (slurm_quick_start_tutorial.html).

1.3.1 - Accounting Configurations

Option	Example	Description
Job Name	`#SBATCH --job-name=<job_name>`	Give your job a short, descriptive name.
Comment	`#SBATCH --comment=<comment>`	Give your job an extended description.
Account	`#SBATCH --account=<account_name>`	Tell Slurm which Slurm account to use.
Partition	`#SBATCH --partition=<debug,tier3>`	Tell Slurm which partition to use.
Time Limit	`#SBATCH --time=D-HH:MM:SS`	Tell Slurm a max time limit for your job.

Reminder: The `debug` partition is ONLY for debugging. We reserve the right to cancel jobs running on the `debug` partition if we need to train a new researcher or help a researcher debug their jobs. DO NOT run production jobs on the `debug` partition.

1.3.2 - Job Output Configurations

Option	Example	Description
Output File	`#SBATCH --output=%x_%j.out`	Where to save output from your job.
Error File	`#SBATCH --error=%x_%j.err`	Where to save errors from your job.

Reminder: If you place your output or error files in a folder, you need to make sure that folder exists before your submit your job. Otherwise, those files may not get saved, or your job may not even schedule.

1.3.3 - Slack Configurations

Option	Example	Description
Slack Username	`#SBATCH --mail-user=slack:abc1234`	The slack username to send notifications to.
Notification Type	`#SBATCH --mail-type=<BEGIN,END,FAIL,ALL>`	The types of notifications to send.

Reminder: If you submit a large number of jobs, make sure they **do not** send notifications. Sending too many notifications at once will spam your slack client.

1.3.4 - Node Configurations

Option	Example	Description
Nodes	`#SBATCH --nodes=<num_nodes>`	The number of nodes your job needs.
Excluding Nodes	`#SBATCH --exclude=<node1,node2,...>`	List nodes your job should NOT run on.
Exclusive Access to a Node	`#SBATCH --exclusive`	Make sure your job is the only one running on a node.

1.3.5 - Task Configurations

Option	Example	Description
Number of Tasks	`#SBATCH --ntasks=<num_tasks>`	Number of tasks (*i.e.* processes). Default=1.
Number of Tasks per Node	`#SBATCH --ntasks-per-node=<num_tasks>`	Number of tasks (*i.e.* processes) per node.

1.3.6 - CPU & GPU Configurations

Option	Example	Description
CPUs per Task	`#SBATCH --cpus-per-task=<num_cpus>`	Number of CPUs per task. Default=1.
Requesting GPUs	`#SBATCH --gres=gpu:<type>:<number>` `$ sinteractive --gres=gpu:<number>`	The type and number of GPUs you need.

1.3.7 - Memory Configurations

Option	Example	Description
Memory per Node	`#SBATCH --mem=<number><k,m,g,t>`	Amount of memory (RAM) your job needs.
Memory per CPU	`#SBATCH --mem-per-cpu=<number><k,m,g,t>`	Amount of memory (RAM) per CPU.
All Memory on a Node	`#SBATCH --mem=0`	Use all available memory on a node.

Reminder: `#SBATCH --mem=0` should be used in conjunction with `#SBATCH --exclusive`.

2 - Serial vs. Parallel Computing

When computers first became a thing, they all ran in **serial**, meaning all instructions given to a processor ran one-at-a-time in sequence. But now, most computers have multiple processors and can run instructions in **parallel**. Typically, parallel compute is much faster than serial compute, but there can be some overhead in parallel computing.

Most programming languages have some built in parallelization that you can use to speed up your code by running multiple computations at the same time on multiple CPUs, such as Python's multiprocessing library. However, built-in multiprocessing libraries are usually restricted to a single node. Fortunately, Message Passing Interface (MPI) allows processes on multiple nodes to communicate with each other. We'll talk more about MPI in the next section.

There are several ways a **parallel job** (one whose tasks run simultaneously) can be created:

- by running a multi-process program (SPMD paradigm, *e.g.* with MPI)
- by running a multithreaded program (shared memory paradigm, *e.g.* with OpenMP or pthreads)
- by running several instances of a single-threaded program (so-called *embarrassingly parallel* paradigm or a *job array*)
- by running one parent program controlling several child programs (*parent/child* paradigm)

Now, parallel programs can use multiple **processes** or multiple **threads**. A process is just a running program. Each process typically has its own private memory. Threads exist within a process, and a process can have one or more threads. Threads share the resources allocated to their parent process.

In the context of Slurm, a task is just a process, thus a multiprocess job has several tasks. By contrast, a multithreaded job has a single task, but that task has threads running on multiple CPUs. You can configure your job to use multiple processes using `#SBATCH --ntasks`, and you can configure your job to use multiple threads per process using `#SBATCH --cpus-per-task`.

Individual tasks cannot be split across multiple compute nodes; requesting multiple CPUs with `#SBATCH --cpus-per-task` will ensure all CPUs for a task are allocated on the same node. By contrast, if you request the same number of CPUs as tasks (*e.g.* `#SBATCH --ntasks=4` and `#SBATCH --cpus-per-task=1`), your CPUs may be allocated on several different nodes.

In the next section, we will go through some examples of different kinds of parallel jobs.

3 - Configuring Parallel Slurm Jobs

3.1 - Multiple Tasks on Multiple Nodes w/ MPI

Message Passing Interface (MPI) is a computing standard which defines how processes on different nodes can communicate with each other. You are responsible for telling your code to use MPI, and there are MPI implementations for most programming languages, such as mpi4py for Python.

Now, let's grab an example C program that uses MPI from wikipedia. Assuming this program is named `hello.c`, we can compile it to use MPI with:

```
$ spack load openmpi
$ mpicc hello.c -o hello.mpi
```

Our sbatch script for running this program might look like this:

```
#!/bin/bash -l
#SBATCH --job-name=simple_mpi   # Name of your job
#SBATCH --account=rc-help       # Your Slurm account
#SBATCH --partition=tier3       # Run on tier3
#SBATCH --output=%x_%j.out      # Output file
#SBATCH --error=%x_%j.err       # Error file
#SBATCH --time=0-00:10:00       # 10 minute time limit
#SBATCH --ntasks=4              # 4 tasks (i.e. processes)
#SBATCH --mem-per-cpu=1g        # 1GB RAM per CPU

spack load openmpi

srun hello.mpi
```

This example requests four CPUs (default of 1 per task), with 1GB of memory per CPU. It also has a 10 minute time limit. If we submit this job and it runs, the output should look similar to this:

```
We have 4 processes.
Process 1 reporting for duty.
Process 2 reporting for duty.
Process 3 reporting for duty.
```

Note that the order the processes respond in might not be the same when you run it.

> **Note:** The `srun` command used above is a special command that tells Slurm to run your code in parallel.

3.2 - Shared Memory Example

Now, what if we need to share memory between CPUs? That's where threads come in and we need to use OpenMP (in the case of C).

Let's grab an example *hello world* program in C from wikipedia and call it `hello.c`. Now we can compile it like this:

```
$ spack load gcc
$ gcc -fopenmp hello.c -o hello.omp
```

Now, we can run this code in parallel and with shared memory by requesting `#SBATCH --ntasks=1` and `#SBATCH --cpus-per-task=4` with the following sbatch script:

```
#!/bin/bash -l
#SBATCH --job-name=shared_mem   # Name of your job
#SBATCH --account=rc-help       # Your Slurm account
#SBATCH --partition=tier3       # Run on tier3
#SBATCH --output=%x_%j.out      # Output file
#SBATCH --error=%x_%j.err       # Error file
#SBATCH --time=0-00:10:00       # 10 minute time limit
#SBATCH --ntasks=1              # 1 tasks (i.e. processes)
#SBATCH --cpus-per-task=4       # 4 CPUs per task
#SBATCH --mem-per-cpu=1g        # 1GB RAM per CPU

# This gets set automatically behind-the-scences; you don't need to set this
# export OMP_NUM_THREADS=$SLURM_CPUS_PER_TASK

spack load gcc

srun hello.omp
```

If we submit the job, the output should look like this:

```
Hello, World!
Hello, World!
Hello, World!
Hello, World!
```

Note the `$SLURM_CPUS_PER_TASK` environment variable. This is set by Slurm and allows you to programmatically access the `--cpus-per-task` that you set. See Section 4 of this tutorial for more details on Slurm Environment Variables.

3.3 - Embarassingly Parallel Example

Many (but not all) compute jobs are what we call *embarassingly parallel*, meaning that each parallel process doesn't care about the other parallel processes (*i.e.* tasks).

For example, consider a program that needs 10,000 randomly drawn samples. You could run this in serial, but that might be slow if drawing a sample is slow. Alternatively, you could break up your problem and run 10 processes in parallel with each process drawing 1,000 random samples; then write those samples to disk and combine them later with another process. (If you've heard of Monte-Carlo simulations, this should sound similar to you.)

Or consider a program that runs the same code many times, but each run differs by some initial value (such as a learning rate in a machine learning model). In this example, you could iterate over every learning rate that you're interested in and train each model (with a different learning rate) sequentially, but this could be brutally slow. Instead, you could redesign your code to take the learning rate as a command line argument, and implement what we call a **parameter sweep** using Slurm.

3.3.1 - Basic Job Array

Consider the following python code, which computes 3 random numbers based on an initial seed:

```
$ cat random_test.py
#!/usr/bin/env python3

import random
import sys

if __name__ == "__main__":
    args = sys.argv[1:]
    seed = args[0]
    print("SEED: {}".format(seed))

    random.seed(seed)
    for i in range(0, 3):
        print(random.random())
```

Now let's say we want our seeds to be 1-8. Here's what this parameter sweep would look like in an sbatch script:

```
#!/bin/bash -l
#SBATCH --job-name=param_sweep  # Name of your job
#SBATCH --account=rc-help       # Your Slurm account
#SBATCH --partition=tier3       # Run on tier3
#SBATCH --output=%x_%j_%a.out   # Output file
#SBATCH --error=%x_%j_%a.err    # Error file
#SBATCH --time=0-00:10:00       # 10 minute time limit
#SBATCH --ntasks=1              # 1 tasks (i.e. processes)
#SBATCH --mem-per-cpu=1g        # 1GB RAM per CPU

#SBATCH --array=1-8

spack load python@3.8.7

srun python3 random_test.py $SLURM_ARRAY_TASK_ID
```

If we submit that job, we should get output similar to this:

```
SEED: 8
0.22991307664292304
0.7963928625032808
0.7965374772142675
...
SEED: 5
0.4782479962566343
0.044242767098090496
0.11703586901195051
```

But what's actually going on here? Notice the `#SBATCH --array=1-8`. This tells Slurm to execute a **job array**, basically to execute many similar jobs. In this case, Slurm will launch 8 jobs in the array, each with a unique index between 1 and 8. Now, we can leverage this to feed different random seeds to each run of `random_test.py` using the `$SLURM_ARRAY_TASK_ID` environment variable. For each run, `$SLURM_ARRAY_TASK_ID` will have a different value between 1 and 8, and we can pass that to `random_test.py` as a random seed.

3.3.2 - Job Array with Array Values

The above example can be quite powerful, but what if we need to use values that aren't sequential, or values that aren't numerical? Well, we can still use `#SBATCH --array`, but we also need to use an array (or list) of values.

Let's say we have a directory with eight books from Project Gutenberg in it and we want to find the number of words in each book:

```
$ ls ~/books/
alice_in_wonderland.txt
count_of_monte_cristo.txt
dracula.txt
frankenstein.txt
moby_dick.txt
pride_and_prejudice.txt
sherlock_holmes.txt
tale_of_two_cities.txt
```

We can do that by using a job array with a list of books, like this:

```
#!/bin/bash -l
#SBATCH --job-name=par_filedir  # Name of your job
#SBATCH --account=rc-help       # Your Slurm account
#SBATCH --partition=tier3       # Run on tier3
#SBATCH --output=%x_%j_%a.out   # Output file
#SBATCH --error=%x_%j_%a.err    # Error file
#SBATCH --time=0-00:10:00       # 10 minute time limit
#SBATCH --ntasks=1              # 1 tasks (i.e. processes)
#SBATCH --mem-per-cpu=1g        # 1GB RAM per CPU

#SBATCH --array=0-7

FILES=(~/books/*)

srun wc -c ${FILES[$SLURM_ARRAY_TASK_ID]}
```

In the above example, we're using `#SBATCH --array=0-7` to say we want eight jobs in the array, with indexes between 0 and 7. Then, we can use those indexes to access each element in the `FILES` array.

We could do the same thing with any kind of array, such as a list of numbers:

```
# SBATCH configurations here
# Spack loads here
ARGS=(0.125 0.250 0.375 0.500 0.625 0.750 0.875 1.000)
srun python3 square.py ${ARGS[$SLURM_ARRAY_TASK_ID]}
```

Warning: If the running time for your code is small, say less than ten minutes, there will be a lot of overhead, and you should consider packing your jobs.

3.4 - Packed Jobs Example

The `srun` command has a (rather counter-inituitevely-named) argument `--exclusive`, which allows scheduling independent processes inside of a Slurm job allocation. As the documentation states:

> This option can also be used when initiating more than one job step within an existing resource allocation, where you want separate processors to be dedicated to each job step. If sufficient processors are not available to initiate the job step, it will be deferred. This can be thought of as providing a mechanism for resource management to the job within it's allocation.

As an example, the following sbatch script requests 8 CPUs. Then it runs the `random_test.py` script 1000 times, each time passing an integer from 1 to 1000:

```
#!/bin/bash -l
#SBATCH --job-name=packed_job   # Name of your job
#SBATCH --account=rc-help       # Your Slurm account
#SBATCH --partition=tier3       # Run on tier3
#SBATCH --output=%x_%j.out      # Output file
#SBATCH --error=%x_%j.err       # Error file
#SBATCH --time=0-00:10:00       # 10 minute time limit
#SBATCH --ntasks=8              # 1 tasks (i.e. processes)
#SBATCH --mem-per-cpu=1g        # 1GB RAM per CPU

spack load python@3.8.7

for i in {1..1000}; do
    srun --nodes=1 --ntasks=1 --cpus-per-task=1 python3 random_test.py $i &
done

wait
```

BUT WAIT! We have a problem: we asked for 8 tasks (thus 8 CPUs), but we're trying to launch 1000 processes. If we submit this job as is, it won't work. We need to use `srun --exclusive` to tell Slurm to launch each of those 1000 processes only when a CPU is available. That looks like this:

```
#!/bin/bash -l
#SBATCH --job-name=packed_job   # Name of your job
#SBATCH --account=rc-help       # Your Slurm account
#SBATCH --partition=tier3       # Run on tier3
#SBATCH --output=%x_%j.out      # Output file
#SBATCH --error=%x_%j.err       # Error file
#SBATCH --time=0-00:10:00       # 10 minute time limit
#SBATCH --ntasks=8              # 1 tasks (i.e. processes)
#SBATCH --mem-per-cpu=1g        # 1GB RAM per CPU

spack load python@3.8.7

for i in {1..1000}; do
    srun --nodes=1 --ntasks=1 --cpus-per-task=1 --exclusive python3 random_test.py $i &
done

wait
```

You can think of the `srun --exclusive` as a sort of mini-scheduler inside a Slurm job. If you have 8 CPUs available, and 8 processes you want to run, `srun` will happily run all 8. However, if you have more than 8 processes you want to run, `srun` will wait until a CPU is available to launch another process.

Note: As you can see, you can also pass Slurm configurations to the `srun` command, which can come in handy in certain scenarios.

3.5 - Parent/Child Example

You might find yourself in a situation where you need one process (a parent) to orchestrate the activity of other processes (children). You may be able to do so within your programming language, or you may need to use `srun --multi-prog`, which you will see in this example.

Consider the following sbatch script:

```
#!/bin/bash -l
#SBATCH --job-name=parent_trap  # Name of your job
#SBATCH --account=rc-help       # Your Slurm account
#SBATCH --partition=tier3       # Run on tier3
#SBATCH --output=%x_%j.out      # Output file
#SBATCH --error=%x_%j.err       # Error file
#SBATCH --time=0-00:10:00       # 10 minute time limit
#SBATCH --ntasks=4              # 4 tasks (i.e. processes)
#SBATCH --mem-per-cpu=1g        # 1GB RAM per CPU

srun --multi-prog multi.conf
```

What will happen here is that Slurm will launch four tasks (processes) based on the configuration in the `multi.conf` file:

```
$ cat multi.conf
0      echo I am the Parent
1-3    echo I am child %t
```

What the `multi.conf` file is saying is:

- launch a process with task number 0, which will run `echo I am the Parent`
- launch three child processes with task numbers 1-3, which will run `echo I am child %t`
    - In this case, `%t` gets filled in with the task number

Now, this is a simple example, but you could use `srun --multi-prog` to run a python script (for example) that takes the task number in as an argument, and performs different tasks based on the task number.

3.6 - Hybrid Jobs

You can also mix *multiprocessing* (MPI) with *multithreading* (OpenMP) in the same job. Here's an example:

```
#!/bin/bash -l
#SBATCH --job-name=hybrid       # Name of your job
#SBATCH --account=rc-help       # Your Slurm account
#SBATCH --partition=tier3       # Run on tier3
#SBATCH --output=%x_%j.out      # Output file
#SBATCH --error=%x_%j.err       # Error file
#SBATCH --time=0-00:10:00       # 10 minute time limit
#SBATCH --ntasks=4              # 4 tasks (i.e. processes)
#SBATCH --cpus-per-task=2       # 2 CPUs per task
#SBATCH --mem-per-cpu=1g        # 1GB RAM per CPU

# This gets set automatically behind-the-scenes; you don't need to set this
# export OMP_NUM_THREADS=$SLURM_CPUS_PER_TASK

spack load openmpi

srun hello.mpi
```

The example above should look familiar, since it's basically the same as the example in Section 3.1 of this tutorial, except that instead of the default 1 CPU per task, now each task has 2 CPUs available to it. **Note:** You are still responsible for making sure your code can use multiple CPUs.

4 - Slurm Environment Variables

When you submit an sbatch script, many of the configurations you provide are accessible with environment variables. There are many SLURM environment variables, but here are the ones you will find most useful:

Slurm Variable	Description
`$SLURM_ARRAY_TASK_ID`	Task ID for job arrays. See Section 3.3 of this tutorial for more details.
`$SLURM_CPUS_PER_TASK`	Number of CPUs per task, which you likely set with `--cpus-per-task`.
`$SLURM_JOB_ID`	Unique ID for the job; useful for naming files that your code creates.
`$SLURM_JOB_NAME`	Name of your job; useful for naming files that your code creates.
`$SLURM_MEM_PER_CPU`	Amount of RAM per CPU, which you likely set with `--mem-per-cpu`.
`$SLURM_MEM_PER_NODE`	Amount of RAM per node, which you likely set with `--mem`.

Slurm Variable	Description
`$SLURM_NTASKS`	Number of tasks (processes), which you likely set with `--ntasks`.
`$SLURM_NTASKS_PER_NODE`	Number of tasks (processes) per node, which you likely set with `--ntasks-per-node`.

Here is a full list of SLURM Output Variables. There are also Slurm Input Variables, but we do not recommend you use them because they **overwrite** your `#SBATCH` configurations.

5 - Miscellaneous Slurm Workloads

5.1 - Heterogeneous Jobs

What if you need to request different resources for different pieces of your job? We recommend you break up your job into multiple smaller jobs, but if you can't do that, Slurm supports **heterogeneous** jobs:

```
#!/bin/bash -l
#SBATCH --job-name=het_job      # Name of your job
#SBATCH --account=rc-help       # Your Slurm account
#SBATCH --partition=tier3       # Run on tier3
#SBATCH --output=%x_%j.out      # Output file
#SBATCH --error=%x_%j.err       # Error file
#SBATCH --time=0-00:10:00       # 10 minute time limit

# Resources for first group:
#SBATCH --ntasks=1 --mem-per-cpu=4g # 1 tasks, 4GB of RAM per task

# Separator for groups:
#SBATCH hetjob

# Resources for second group:
#SBATCH --ntasks=2 --mem-per-cpu=2g # 2 tasks, 2GB of RAM per task

spack load python@3.8.7

# Script for first group:
srun --pack-group 0 some_script.py

# Script for second group:
srun --pack-group 1 some_other_script.py
```

Let's break that down:

- `#SBATCH hetjob` is a special separator that tells Slurm that the following configurations are for a different group
- `--pack-group <num>` is a flag that tells `srun` which resources to use
- Configurations that don't get redefined after a `#SBATCH hetjob` get carried over to the next group

You can also use Slurm environment variables in heterogeneous jobs, but you need to append `_PACK_GROUP_<num>` to the variable you want to use. For example:

```
srun --pack-group 0 some_script.py $SLURM_NTASKS_PACK_GROUP_0
srun --pack-group 1 some_other_script.py $SLURM_NTASKS_PACK_GROUP_1
```

Note: We have only seen a handful of workflows over the past 20 years where heterogeneous jobs make sense. We include an example here because it might be beneficial in rare cases, but it is very unlikely that you need to use heterogeneous jobs.

5.2 - Job Looping

Researchers typically do not submit a single job, wait for it to complete, look at the results, and call it a day. More often, researchers have a compute workload (*e.g.* model training, simulation) that they think is feasible and want to run it under (many) different conditions.

For example, imagine you have a model that (you think) predicts the weather, and you have 10 weeks of weather data. You can write a script that submits 10 Slurm jobs, each working with a single week of data.

Or maybe you have a model that simulates a black hole. You could write a script that submits 1000 Slurm jobs, each performing a simulation with a different initial value for mass.

Let's look a more abstract example that loops over two input parameters, alpha and beta. Let's look at this `slurm_payload.sh` file:

```
#!/bin/bash -l
#SBATCH --account=rc-help       # Your Slurm account
#SBATCH --partition=tier3       # Run on tier3
#SBATCH --output=%x_%j.out      # Output file
#SBATCH --error=%x_%j.err       # Error file
#SBATCH --time=0-00:10:00       # 10 minute time limit
#SBATCH --ntasks=1              # 1 tasks (i.e. processes)
#SBATCH --mem-per-cpu=1g        # 1GB RAM per CPU

spack load python@3.8.7

echo "Job Name:" $SLURM_JOB_NAME
python3 run_simulation.py $alpha $beta
```

This should look familiar, except we haven't provided a `--job-name`, and where did the `$alpha` and `$beta` variables come from?

Let's look at the more interesting `submit_many_jobs.sh` file:

```
#!/bin/bash -l

# Name of the sbatch script we want to submit many times
jobfile="slurm_payload.sh"

# Values for alpha and beta
alpha_values=(0.0 0.25 0.5 0.75 1.0)
beta_values=(0 1 2 4 8)

echo
echo "Preparing to submit many jobs..."
echo

# For each value of alpha we want to test
for alpha in "${alpha_values[@]}"; do
    # For each value of beta we want to test
    for beta in "${beta_values[@]}"; do
        # Give your job a meaningful name
        jobname=test_$alpha_$beta

        # Export alpha and beta as environment variables
        export alpha
        export beta

        # Submit the job
        sbatch --job-name=$jobname $jobfile
    done
done

echo
echo "Done submitting many jobs!"
```

This is just a standard bash script, not an sbatch script. If we ran `$ bash submit_many_jobs.sh`, the script would loop over the values of alpha and beta that we want to test, export those values to environment variables so `slurm_payload.sh` can use them, and then we submit a job with a unique `--job-name`.

Note that the `--job-name` we provided contains `$alpha` and `$beta` so the output and error files will always have those values in them (because we used `%x` in `slurm_payload.sh`).

If we actually run `$ bash submit_many_jobs.sh`, we would get the following output:

```
Preparing to submit many jobs

test_0.0_0
test_0.0_1
test_0.0_2
test_0.0_4
test_0.0_8
...
test_1.0_0
test_1.0_1
test_1.0_2
test_1.0_4
test_1.0_8

Done submitting many jobs!
```

If we ran `squeue --me`, we would see that 25 jobs were submitted:

```
[abc1234@sporcsubmit ~/]$ squeue --me
  JOBID PARTITION     NAME     USER  ST       TIME  NODES NODELIST(REASON)
    736     debug test_0.0   abc1234  PD       0:00      1 (Resources)
    737     debug test_0.0   abc1234  PD       0:00      1 (Priority)
    738     debug test_0.0   abc1234  PD       0:00      1 (Priority)
    739     debug test_0.0   abc1234  PD       0:00      1 (Priority)
    740     debug test_0.0   abc1234  PD       0:00      1 (Priority)
    741     debug test_0.2   abc1234  PD       0:00      1 (Priority)
    742     debug test_0.2   abc1234  PD       0:00      1 (Priority)
    743     debug test_0.2   abc1234  PD       0:00      1 (Priority)
    744     debug test_0.2   abc1234  PD       0:00      1 (Priority)
    745     debug test_0.2   abc1234  PD       0:00      1 (Priority)
    746     debug test_0.5   abc1234  PD       0:00      1 (Priority)
    747     debug test_0.5   abc1234  PD       0:00      1 (Priority)
    748     debug test_0.5   abc1234  PD       0:00      1 (Priority)
    749     debug test_0.5   abc1234  PD       0:00      1 (Priority)
    750     debug test_0.5   abc1234  PD       0:00      1 (Priority)
    751     debug test_0.7   abc1234  PD       0:00      1 (Priority)
    752     debug test_0.7   abc1234  PD       0:00      1 (Priority)
    753     debug test_0.7   abc1234  PD       0:00      1 (Priority)
    754     debug test_0.7   abc1234  PD       0:00      1 (Priority)
    755     debug test_0.7   abc1234  PD       0:00      1 (Priority)
    756     debug test_1.0   abc1234  PD       0:00      1 (Priority)
    734     debug test_1.0   abc1234   R       0:01      1 bach
    735     debug test_1.0   abc1234   R       0:01      1 tesla
    732     debug test_1.0   abc1234   R       0:05      1 curie
    733     debug test_1.0   abc1234   R       0:05      1 einstein
```

Four of our jobs are running, the rest are in a pending state. If we run `ls`, we would see that our ourput and error files have been created for the currently running jobs:

```
drwxrwx--- 2 abc1234 abc1234 2.0K Jan 15 10:39 .
drwxr-x--- 3 abc1234 abc1234 2.0K Jan 15 10:38 ..
-rw-rw---- 1 abc1234 abc1234  349 Jan 15 10:39 test_1.0_0.out
-rw-rw---- 1 abc1234 abc1234  349 Jan 15 10:39 test_1.0_0.err
...
-rw-rw---- 1 abc1234 abc1234  349 Jan 15 10:39 test_1.0_8.out
-rw-rw---- 1 abc1234 abc1234  349 Jan 15 10:39 test_1.0_8.err
```

5.3 - Job Dependencies

Maybe your workload can be broken up into a bunch of sequential compute jobs, each job needing to run after the previous one finishes. For example:

- Job 1: Collect data
- Job 2: Clean up data
- Job 3: Train a model with the data
- Job 4: Perform classification with the trained model

Those steps need to be performed in that order. Fortunately, we can submit all of these jobs and use the `--dependency` flag to tell Slurm that Job 2 depends on Job 1, Job 3 depends on Job 2, and Job 4 depends on Job 3.

First, let's look at an sbatch script, `slurm_payload.sh`:

```
#!/bin/bash -l
#SBATCH --account=rc-help       # Your Slurm account
#SBATCH --partition=tier3       # Run on tier3
#SBATCH --output=%x_%j.out      # Output file
#SBATCH --error=%x_%j.err       # Error file
#SBATCH --time=0-00:10:00       # 10 minute time limit
#SBATCH --ntasks=1              # 1 tasks (i.e. processes)
#SBATCH --mem-per-cpu=1g        # 1GB RAM per CPU

spack load python@3.8.7

python3 classifier.py $step
```

Let's look at `submit_dependent_jobs.sh` which demonstrates how you can use the `--dependency` flag:

```
#!/bin/bash -l

# Arguments to tell classifier.py what to do
steps=("collect" "clean" "train" "classify")

echo
echo "Preparing to submit dependent jobs..."
echo

# We will use this variable to keep track of the last job we submitted
latest_id=0

# For each step
for step in "$steps[@]}"; do
    # Export $step so slurm_payload.sh can use it
    export $step

    # If this is the first job
    if [[ $step == "collect" ]]; then
        # Put the sbatch command in a variable just for readability
        command="sbatch --job-name=step_$step slurm_payload.sh"
    # If this isn't the first job
    else
        # Put the sbatch command in a variable just for readability
        command="sbatch --job-name=step_$step --dependency=afterok:$latest_id slurm_payl
oad.sh"
    fi

    # Submit the job and grab its job ID
    latest_id=$($command | awk ' { print $4 }')
done

echo
echo "Done submitting dependent jobs!"
```

So, here's what happens when we run `$ bash submit_dependent_jobs.sh`. For every value of `$steps`:

1. First we `export $step` so `slurm_payload.sh` can use it.
2. Next we determine if we need to submit the first job (which has no dependency). If we do, then we just run a typical `sbatch` command, but we need to wrap it in an `awk` command so we can grab the job ID and store it in `$latest_id`.
3. If we need to submit a job with a dependency, then we tell `sbatch` that we have a `--dependency` on the job with the `$latest_id` (we'll discuss `afterok` in a moment). Again, we wrap the `sbatch` command in an `awk` command so we can grab the job ID and store it in `$latest_id`.

Now, there are a few different ways to tell Slurm when to run a job with a dependency. We won't discuss them all, but here are a few that you may find useful:

Condition	Example	Description
`afterany`	`--dependency=afterany:<job_id>`	Run the job after its dependent job finishes (success or failure)
`afterok`	`--dependency=afterok:<job_id>`	Run the job after its dependent job finishes successfully
`afternotok`	`--dependency=afternotok:<job_id>`	Run the job after its dependent job fails

If you just specify `--dependency=<job_id>`, the default condition is `afterany`.

6 - CPUs vs GPUs

Central Processing Units (CPUs) and Graphical Processing Units (GPUs) have a lot in common, but they also have some key differences. CPUs are basically the brains of computers. Typically, all of the commands, processes, and programs that you run on a computer get executed on the CPU. However, CPUs can be a bottleneck for high performance computing workflows. Both CPUs and GPUs have multiple cores, but the cores on a GPU are more specialized and designed to work together more closely. When a task can be divided up and processed across many cores, GPUs can be much more performant than CPUs.

Notice how we keep saying **can be** up there; depending on your research and the nature of your data/computations, GPUs might not be able to speed up your compute jobs. If you're not sure, don't fret, we are happy to help you determine what compute resources you need.

7 - Asking Slurm for GPUs

Some nodes in the SPORC cluster have GPUs. You can see which nodes have what kinds of GPUs with a simple `sinfo` command:

```
$ sinfo -o "%P %.10G %N"
```

> **Warning:** We have a limited number of GPUs and everyone wants to use them. It's important to make sure that the GPUs you request are actually being used by your code. If you have idle GPUs, no one else can use them until your job finishes running. We reserve the right to cancel jobs with idle GPUs.

By default, Slurm will not allocate a GPU for you, so you need to use the `#SBATCH --gres` configuration option to tell Slurm that you need a GPU:

```
#!/bin/bash -l
#SBATCH --job-name=simple_gpu   # Name of your job
#SBATCH --account=rc-help       # Your Slurm account
#SBATCH --partition=tier3       # Run on tier3
#SBATCH --output=%x_%j.out      # Output file
#SBATCH --error=%x_%j.err       # Error file
#SBATCH --time=0-00:10:00       # 10 minute time limit
#SBATCH --ntasks=1              # 1 tasks (i.e. processes)
#SBATCH --mem=1g                # 1GB RAM
#SBATCH --gres=gpu:a100:1       # 1 a100 GPU

./gpu_burn 600
```

In this example, we asked for 1 a100 GPU, but we can ask for more. The format for `--gres` is `#SBATCH --gres=gpu:<type>:<number>`, where `<type>` is the type of GPU you want (currently, only a100's are available), and `<number>` is how many GPUs you want.

If you don't care what kind of GPU you get, you can use `#SBATCH --gres=gpu:<number>` and the first available GPU(s) will be allocated to you. **Note:** Different GPUs have different amounts of memory, so you want to make sure that your code can handle that if you're asking for an arbitrary GPU type.

You can also use `--gres` with `sinteractive` to request GPUs for your interactive jobs.

Note: Even if you request a GPU, you need to make sure your code can use it. Many machine learning frameworks have the ability to leverage GPUs, but you typically need to tell those libraries to do so.

8 - Conclusions

In this tutorial, you learned:

- The difference between serial and parallel computing.
- The basics of processes vs. threads.
- Some different ways to configure parallel Slurm jobs, including:
    - Multiprocessing jobs
    - Multithreading jobs
    - Embarassingly parallel workloads with Slurm job arrays
    - Packed jobs
    - Parents/child jobs
    - Hybrid multiprocessing/multithreading jobs
- Some special Slurm workloads, including:

    - Job dependencies
    - Heterogeneous jobs
- The difference between CPUs and GPUs.
- How to request GPUs in sbatch scripts.
- How to use Slurm environment variables.

Help: If there are any further questions, or there is an issue with the documentation, please **submit a ticket** or contact us on Slack for additional assistance.

Tags: instructions (tag_instructions.html) slurm (tag_slurm.html)

Internal Documentation

Page last updated: February 9, 2023

Site last generated: Mar 19, 2026